

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Muhammad Bukhari Fitri NIM. 2310817210015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Bukhari Fitri
NIM : 2310817210015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program	27
C. Pembahasan	31
D. Tautan Git	56

DAFTAR GAMBAR

Gambar 1. Tampilan Layar Detail.....	27
Gambar 2. Tampilan Awal	28
Gambar 3. Tampilan Awal Landscape	29
Gambar 4. Tampilan Awal Dark Mode	31

DAFTAR TABEL

Table 1. Source Code presentation/MainActivity.kt	6
Table 2. Source Code AppContainer.kt.....	8
Table 3. Source Code MyApplication.kt	9
Table 4. Source Code core/common/NetworkResult	10
Table 5. Source Code data/local/dao/MovieDao.kt.....	10
Table 6. Source Code data/local/model/MovieEntity.kt	11
Table 7. Source Code data/local/AppDatabase.kt	11
Table 8. Source Code Data/mapper/MovieMapper.kt.....	12
Table 9. Source Code Data/remote/api/ApiConstants.kt.....	12
Table 10. Source Code data/remote/api/KtroApiClient.kt	13
Table 11. Source Code data/remote/model/MovieDto.kt	13
Table 12. Source Code data/repository/MovieRepositoryImpl.kt.....	14
Table 13. Source Code domain/model/Movie.kt.....	15
Table 14. Source Code domain/repository/MovieRepository.kt.....	16
Table 15. Source Code Domain/usecase/GetMovieUseCase.kt.....	16
Table 16. Source Code presentation/ui/component/MovieCard.kt	17
Table 17. Source Code presentation/ui/navigation/Screen.kt.....	19
Table 18. Source Code presentation/ui/screens/detail/MovieDetailScreen.kt.....	19
Table 19. Source Code presentation/ui/screens/list/MovieListScreen.kt	21
Table 20. Source Code presentation/ui/screens/list/MovieListViewModel.kt	24
Table 21. Source Code presentation/ui/screens/list/MovieListViewModelFactory.kt.....	26

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
<https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

presentation/MainActivity.kt

Table 1. Source Code presentation/MainActivity.kt

1	package com.example.movielist.presentation
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.isSystemInDarkTheme
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.material3.MaterialTheme
9	import androidx.compose.material3.Surface
10	import androidx.compose.material3.Text
11	import androidx.compose.runtime.Composable
12	import androidx.compose.runtime.collectAsState
13	import androidx.compose.runtime.getValue
14	import androidx.compose.ui.Modifier

```

15 import androidx.lifecycle.viewmodel.compose.viewModel
16 import androidx.navigation.compose.NavHost
17 import androidx.navigation.compose.composable
18 import androidx.navigation.compose.rememberNavController
19 import androidx.navigation.navArgument
20 import androidx.navigation.NavType
21 import com.example.movielist.AppContainer
22 import com.example.movielist.domain.model.Movie
23 import
  com.example.movielist.presentation.ui.screens.detail.MovieDetailScreen
24 import
  com.example.movielist.presentation.ui.screens.list.MovieListScreen
25 import
  com.example.movielist.presentation.ui.screens.list.MovieListViewModel
26 import com.example.movielist.ui.theme.MovieListTheme
27 import com.google.gson.Gson
28 class MainActivity : ComponentActivity() {
29     override fun onCreate(savedInstanceState: Bundle?) {
30         super.onCreate(savedInstanceState)
31
32         val appContainer = AppContainer(applicationContext)
33
34         setContent {
35             val settingsManager =
  appContainer.settingsManager
36             val isSystemDark = isSystemInDarkTheme()
37             // Mengamati isDarkMode dari SettingsManager,
  dengan initial value dari tema sistem
38             val isDarkMode by
  settingsManager.isDarkMode.collectAsState(initial =
  isSystemDark)
39
40             MovieListTheme(darkTheme = isDarkMode) {
41                 Surface(
42                     modifier = Modifier.fillMaxSize(),
43                     color =
  MaterialTheme.colorScheme.background
44                 ) {
45                     val navController =
  rememberNavController()
46                     val movieListViewModel:
  MovieListViewModel = viewModel(
47                         factory =
  appContainer.movieListViewModelFactory
48                     )
49
50                     NavHost(navController = navController,
  startDestination = "movie_list") {
51                         composable("movie_list") {

```

52	MovieListScreen(
53	viewModel =
54	movieListViewModel,
55	settingsManager =
56	settingsManager,
57	navController = navController
58	)
59	composable(
60	route =
61	"movie_detail/{movieJson}",
62	arguments =
63	listOf(navArgument("movieJson") { type = NavType.StringType
64	}) { backStackEntry ->
65	val movieJson =
66	backStackEntry.arguments?.getString("movieJson")
67	val movie =
68	Gson().fromJson(movieJson, Movie::class.java)
69	movie?.let {
70	MovieDetailScreen(movie = it)
71	} ?: run {
72	Text("Error: Movie details
73	not available.", color = MaterialTheme.colorScheme.error)
74	}
75	}

AppContainer.kt

Table 2. Source Code AppContainer.kt

1	package com.example.movielist
2	
3	import android.content.Context
4	import com.example.movielist.data.local.AppDatabase
5	import com.example.movielist.data.remote.api.KtorApiClient
6	import
7	com.example.movielist.data.repository.MovieRepositoryImpl
8	import
9	com.example.movielist.domain.repository.MovieRepository
10	import com.example.movielist.domain.usecase.GetMoviesUseCase
11	import
12	com.example.movielist.presentation.ui.screens.list.MovieListV
13	iewModelFactory
14	import com.example.movielist.util.SettingsManager

11	
12	class AppContainer(private val context: Context) {
13	
14	private val appDatabase: AppDatabase by lazy {
15	AppDatabase.create(context)
16	}
17	
18	private val movieDao by lazy {
19	appDatabase.movieDao()
20	}
21	
22	private val ktorApiClient: KtorApiClient by lazy {
23	KtorApiClient
24	}
25	
26	private val movieRepository: MovieRepository by lazy {
27	MovieRepositoryImpl(ktorApiClient, movieDao)
28	}
29	
30	private val getMoviesUseCase: GetMoviesUseCase by lazy {
31	GetMoviesUseCase(movieRepository)
32	}
33	
34	val movieListViewModelFactory: MovieListViewModelFactory
35	by lazy {
36	MovieListViewModelFactory(getMoviesUseCase)
37	}
38	
39	val settingsManager: SettingsManager by lazy {
40	SettingsManager(context)
41	}

MyApplication.kt

Table 3. Source Code MyApplication.kt

1	package com.example.movielist
2	
3	import android.app.Application
4	
5	class MyApplication : Application() {
6	lateinit var container: AppContainer
7	
8	override fun onCreate() {
9	super.onCreate()
10	container = AppContainer(this)
11	}
12	}

core/common/NetworkResult.kt

Table 4. Source Code core/common/NetworkResult

1	package	com.example.movielist.core.common
2		
3	sealed class	NetworkResult<T> (
4	val data:	T? = null,
5	val message:	String? = null
6	)	{
7	class Success<T>(data: T)	: NetworkResult<T>(data)
8	class Error<T>(message: String, data: T? = null)	: NetworkResult<T>(data, message)
9	class Loading<T>	: NetworkResult<T>()
10	}	

data/local/dao/MovieDao.kt

Table 5. Source Code data/local/dao/MovieDao.kt

1	package	com.example.movielist.data.local.dao
2		
3	import	androidx.room.Dao
4	import	androidx.room.Insert
5	import	androidx.room.OnConflictStrategy
6	import	androidx.room.Query
7	import	androidx.room.Transaction
8	import	com.example.movielist.data.local.model.MovieEntity
9	import	kotlinx.coroutines.flow.Flow
10		
11	@Dao	
12	interface	MovieDao {
13		
14	@Query("SELECT * FROM movies")	
15	fun getMovies():	Flow<List<MovieEntity>>
16		
17	@Insert(onConflict = OnConflictStrategy.REPLACE)	
18	suspend fun insertMovies(movies: List<MovieEntity>)	
19		
20	@Query("DELETE FROM movies")	
21	suspend fun clearMovies()	
22		
23	@Transaction	
24	suspend fun clearAndInsertMovies(movies: List<MovieEntity>)	{
25	clearMovies()	
26	insertMovies(movies)	
27	}	
28	}	

data/local/model/MovieEntity.kt

Table 6. Source Code data/local/model/MovieEntity.kt

1	package com.example.movielist.data.local.model
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "movies")
7	data class MovieEntity(
8	@PrimaryKey val id: Int,
9	val title: String,
10	val overview: String,
11	val posterPath: String,
12	val backdropPath: String,
13	val releaseDate: String,
14	val voteAverage: Double,
15	val popularity: Double
16	)

Data/local/AppDatabase.kt

Table 7. Source Code data/local/AppDatabase.kt

1	package com.example.movielist.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	import com.example.movielist.data.local.dao.MovieDao
8	import com.example.movielist.data.local.model.MovieEntity
9	
10	@Database(entities = [MovieEntity::class], version = 1,
	exportSchema = false)
11	abstract class AppDatabase : RoomDatabase() {
12	abstract fun movieDao(): MovieDao
13	
14	companion object {
15	@Volatile
16	private var INSTANCE: AppDatabase? = null
17	
18	fun create(context: Context): AppDatabase {
19	return INSTANCE ?: synchronized(this) {
20	val instance = Room.databaseBuilder(
21	context.applicationContext,
22	AppDatabase::class.java,
23	"movie_database"
24	)
25	.fallbackToDestructiveMigration()
26	.build()
27	INSTANCE = instance
28	instance

29	}
30	}
31	}
32	}

Data/mapper/MovieMapper.kt

Table 8. Source Code Data/mapper/MovieMapper.kt

1	package com.example.movieslist.data.mapper
2	
3	import com.example.movieslist.data.local.model.MovieEntity
4	import
	com.example.movieslist.data.remote.model.MovieResultDto
5	import com.example.movieslist.domain.model.Movie
6	
7	fun MovieResultDto.toEntity(): MovieEntity {
8	return MovieEntity(
9	id = id ?: 0,
10	title = title.orEmpty(),
11	overview = overview.orEmpty(),
12	posterPath = posterPath.orEmpty(),
13	backdropPath = backdropPath.orEmpty(),
14	releaseDate = releaseDate.orEmpty(),
15	voteAverage = voteAverage ?: 0.0,
16	popularity = popularity ?: 0.0
17	)
18	}
19	
20	fun MovieEntity.toDomain(): Movie {
21	return Movie(
22	id = this.id,
23	title = this.title,
24	overview = this.overview,
25	posterPath = this.posterPath,
26	backdropPath = this.backdropPath,
27	releaseDate = this.releaseDate,
28	voteAverage = this.voteAverage
29	)
30	}

Data/remote/api/ApiConstants.kt

Table 9. Source Code Data/remote/api/ApiConstants.kt

1	package com.example.movieslist.data.remote.api
2	
3	object ApiConstants {
4	const val BASE_URL = "https://api.themoviedb.org/3/"
5	const val API_KEY = "api key saya, disamakan di

6	laprak" }
---	--------------

Data/remote/api/KtorApiClient.kt

Table 10. Source Code data/remote/api/KtroApiClient.kt

1	package com.example.movielist.data.remote.api
2	
3	import
	com.example.movielist.data.remote.model.MovieResponseDto
4	import io.ktor.client.*
5	import io.ktor.client.call.*
6	import io.ktor.client.engine.android.*
7	import io.ktor.client.plugins.contentnegotiation.*
8	import io.ktor.client.plugins.logging.*
9	import io.ktor.client.request.*
10	import io.ktor.serialization.kotlinx.json.json
11	import kotlinx.serialization.json.Json
12	
13	object KtorApiClient {
14	val client = HttpClient(Android) {
15	install(Logging) {
16	level = LogLevel.ALL
17	}
18	install(ContentNegotiation) {
19	json(Json {
20	ignoreUnknownKeys = true
21	isLenient = true
22	prettyPrint = true
23	})
24	}
25	}
26	
27	suspend fun getPopularMovies(): MovieResponseDto {
28	return client.get(ApiConstants.BASE_URL +
	"movie/popular") {
29	parameter("api_key", ApiConstants.API_KEY)
30	}.body()
31	}
32	}

Data/remote/model/MovieDto.kt

Table 11. Source Code data/remote/model/MovieDto.kt

1	package com.example.movielist.data.remote.model
2	
3	import android.annotation.SuppressLint
4	import kotlinx.serialization.SerialName
5	import kotlinx.serialization.Serializable

6	
7	<code>@SuppressLint("UnsafeOptInUsageError")</code>
8	<code>@Serializable</code>
9	<code>data class MovieResponseDto(</code>
10	<code> val page: Int?,</code>
11	<code> val results: List<MovieResultDto>?,</code>
12	<code> @SerializedName("total_pages")</code>
13	<code> val totalPages: Int?,</code>
14	<code> @SerializedName("total_results")</code>
15	<code> val totalResults: Int?</code>
16	<code>)</code>
17	
18	<code>@SuppressLint("UnsafeOptInUsageError")</code>
19	<code>@Serializable</code>
20	<code>data class MovieResultDto(</code>
21	<code> val id: Int?,</code>
22	<code> val title: String?,</code>
23	<code> val overview: String?,</code>
24	<code> @SerializedName("poster_path")</code>
25	<code> val posterPath: String?,</code>
26	<code> @SerializedName("release_date")</code>
27	<code> val releaseDate: String?,</code>
28	<code> @SerializedName("vote_average")</code>
29	<code> val voteAverage: Double?,</code>
30	<code> @SerializedName("backdrop_path")</code>
31	<code> val backdropPath: String?,</code>
32	<code> val popularity: Double?</code>
33	<code>)</code>

Data/repository/MovieRepositoryImpl.kt

Table 12. Source Code data/repository/MovieRepositoryImpl.kt

1	<code>package com.example.movielist.data.repository</code>
2	
3	<code>import com.example.movielist.core.common.NetworkResult</code>
4	<code>import com.example.movielist.data.local.dao.MovieDao</code>
5	<code>import com.example.movielist.data.mapper.toDomain</code>
6	<code>import com.example.movielist.data.mapper.toEntity</code>
7	<code>import com.example.movielist.data.remote.api.KtorApiClient</code>
8	<code>import</code>
	<code>com.example.movielist.data.remote.model.MovieResponseDto</code>
9	<code>import com.example.movielist.domain.model.Movie</code>
10	<code>import</code>
	<code>com.example.movielist.domain.repository.MovieRepository</code>
11	<code>import kotlinx.coroutines.flow.Flow</code>
12	<code>import kotlinx.coroutines.flow.map</code>
13	<code>import java.io.IOException</code>
14	
15	<code>class MovieRepositoryImpl(</code>
16	<code> private val apiClient: KtorApiClient,</code>

17	private val movieDao: MovieDao
18	) : MovieRepository {
19	
20	override fun getMovies(): Flow<List<Movie>> {
21	return movieDao.getMovies().map { entities ->
22	entities.map { it.toDomain() }
23	}
24	}
25	
26	override suspend fun refreshMovies():
27	NetworkResult<Unit> {
28	return try {
29	val response: MovieResponseDto =
30	apiClient.getPopularMovies()
31	val movieEntities = response.results?.map {
32	it.toEntity() } ?: emptyList()
33	movieDao.clearAndInsertMovies(movieEntities)
34	NetworkResult.Success(Unit)
35	} catch (e: IOException) {
36	NetworkResult.Error("Gagal menyambung ke
37	server. Periksa koneksi internet Anda.")
38	} catch (e: Exception) {
	NetworkResult.Error("Terjadi error:
	`\${e.message}`")
	}
	}
	}

Domain/model/Movie.kt

Table 13. Source Code domain/model/Movie.kt

1	package com.example.movieslist.domain.model
2	
3	data class Movie(
4	val id: Int,
5	val title: String,
6	val overview: String,
7	val posterPath: String?,
8	val backdropPath: String?,
9	val voteAverage: Double,
10	val releaseDate: String?
11	) {
12	fun getPosterUrl(): String {
13	return if (posterPath != null &&
14	posterPath.isNotEmpty())
15	"https://image.tmdb.org/t/p/w500\$posterPath" else ""
16	}
17	fun getBackdropUrl(): String {
	return if (backdropPath != null &&

	<code>backdropPath.isEmpty())</code>
	<code>"https://image.tmdb.org/t/p/w780\$backdropPath" else ""</code>
18	<code>}</code>
19	
20	<code>fun getTmdbUrl(): String {</code>
21	<code>return "https://www.themoviedb.org/movie/\$id"</code>
22	<code>}</code>
23	<code>}</code>

Domain/repository/MovieRepository.kt

Table 14. Source Code domain/repository/MovieRepository.kt

1	<code>package com.example.movielist.domain.repository</code>
2	
3	<code>import com.example.movielist.core.common.NetworkResult</code>
4	<code>import com.example.movielist.domain.model.Movie</code>
5	<code>import kotlinx.coroutines.flow.Flow</code>
6	
7	<code>interface MovieRepository {</code>
8	<code>fun getMovies(): Flow<List<Movie>></code>
9	<code>suspend fun refreshMovies(): NetworkResult<Unit></code>
10	<code>}</code>

Domain/usecase/GetMoviesUseCase.kt

Table 15. Source Code Domain/usecase/GetMovieUseCase.kt

1	<code>package com.example.movielist.domain.usecase</code>
2	
3	<code>import com.example.movielist.core.common.NetworkResult</code>
4	<code>import com.example.movielist.domain.model.Movie</code>
5	<code>import</code>
	<code>com.example.movielist.domain.repository.MovieRepository</code>
	<code>import kotlinx.coroutines.flow.Flow</code>
6	
7	<code>class GetMoviesUseCase(private val movieRepository:</code>
8	<code>MovieRepository) {</code>
9	<code>fun getMoviesFromDb(): Flow<List<Movie>> =</code>
	<code>movieRepository.getMovies()</code>
10	
11	<code>suspend fun refreshMoviesFromApi(): NetworkResult<Unit></code>
	<code>= movieRepository.refreshMovies()</code>
12	<code>}</code>

Presentation/ui/component/MovieCard.kt

Table 16. Source Code presentation/ui/component/MovieCard.kt

```

1 package com.example.movielist.presentation.ui.component
2
3 import androidx.compose.foundation.clickable
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.foundation.shape.RoundedCornerShape
6 import androidx.compose.material3.*
7 import androidx.compose.runtime.Composable
8 import androidx.compose.ui.Alignment
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.draw.clip
11 import androidx.compose.ui.layout.ContentScale
12 import androidx.compose.ui.platform.LocalContext
13 import androidx.compose.ui.text.font.FontWeight
14 import androidx.compose.ui.text.style.TextOverflow
15 import androidx.compose.ui.unit.dp
16 import coil.compose.AsyncImage
17 import coil.request.ImageRequest
18 import com.example.movielist.domain.model.Movie
19
20 @OptIn(ExperimentalMaterial3Api::class)
21 @Composable
22 fun MovieCard(
23     movie: Movie,
24     modifier: Modifier = Modifier,
25     onMovieClick: (Movie) -> Unit,
26     onTmdbClick: (String) -> Unit
27 ) {
28     val context = LocalContext.current
29
30     Card(
31         modifier = modifier
32             .fillMaxWidth()
33             .clickable { onMovieClick(movie) }
34             .padding(8.dp),
35         shape = RoundedCornerShape(12.dp),
36         elevation =
37             CardDefaults.cardElevation(defaultElevation = 4.dp)
38     ) {
39         Row(
40             modifier = Modifier
41                 .fillMaxWidth()
42                 .padding(12.dp)
43         ) {
44             AsyncImage(
45                 model =
46                 ImageRequest.Builder(LocalContext.current)
47                     .data(movie.getPosterUrl())
48                     .crossfade(true)
49                     .build(),
50                 contentDescription = movie.title,

```

```

49         modifier = Modifier
50             .size(width = 100.dp, height = 150.dp)
51             .clip(RoundedCornerShape(8.dp)),
52         contentScale = ContentScale.Crop
53     )
54
55     Spacer(modifier = Modifier.width(12.dp))
56
57     Column(
58         modifier = Modifier
59             .weight(1f)
60             .align(Alignment.CenterVertically)
61     ) {
62         Text(
63             text = movie.title,
64             style =
65                 MaterialTheme.typography.titleMedium,
66                 fontWeight = FontWeight.Bold,
67                 maxLines = 2,
68                 overflow = TextOverflow.Ellipsis
69         )
70         Spacer(modifier = Modifier.height(4.dp))
71         Text(
72             text = "Rating: ${String.format("%.1f",
73                 movie.voteAverage)}/10",
74             style =
75                 MaterialTheme.typography.bodySmall
76         )
77         Spacer(modifier = Modifier.height(8.dp))
78
79         Row(
80             modifier = Modifier.fillMaxWidth(),
81             horizontalArrangement = Arrangement.End
82         ) {
83             Button(
84                 onClick = {
85                     onTmdbClick(movie.getTmdbUrl()) },
86                 modifier = Modifier.padding(end =
87                     8.dp)
88             ) {
89                 Text("TMDB")
90             }
91
92             Button(
93                 onClick = { onMovieClick(movie) }
94             ) {
95                 Text("Detail")
96             }
97         }
98     }
99 }

```

96	}
97	}

Presentation/ui/navigation/Screen.kt

Table 17. Source Code presentation/ui/navigation/Screen.kt

1	package com.example.movielist.presentation.ui.navigation
2	
3	sealed class Screen(val route: String) {
4	data object MovieList : Screen("movie_list")
5	data object MovieDetail : Screen("movie_detail")
6	}

Presentation/ui/screens/detail/MovieDetailScreen.kt

Table 18. Source Code presentation/ui/screens/detail/MovieDetailScreen.kt

1	package
2	com.example.movielist.presentation.ui.screens.detail
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.rememberScrollState
5	import androidx.compose.foundation.verticalScroll
6	import androidx.compose.material.icons.Icons
7	import androidx.compose.material.icons.filled.Star
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.ui.Alignment
11	import androidx.compose.ui.Modifier
12	import androidx.compose.ui.draw.clip
13	import androidx.compose.ui.graphics.Color
14	import androidx.compose.ui.layout.ContentScale
15	import androidx.compose.ui.platform.LocalContext
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import coil.compose.AsyncImage
19	import coil.request.ImageRequest
20	import com.example.movielist.domain.model.Movie
21	
22	@Composable
23	fun MovieDetailScreen(movie: Movie) {
24	Column(
25	modifier = Modifier
26	.fillMaxSize()
27	.padding(16.dp)
28	.verticalScroll(rememberScrollState())
29	) {
30	AsyncImage(
31	model =
	ImageRequest.Builder(LocalContext.current)

```

32         .data(movie.getBackdropUrl())
33         .crossfade(true)
34         .build(),
35         contentDescription = movie.title,
36         modifier = Modifier
37             .fillMaxWidth()
38             .height(220.dp)
39             .clip(MaterialTheme.shapes.medium),
40         contentScale = ContentScale.Crop
41     )
42     Spacer(modifier = Modifier.height(16.dp))
43
44     Text(
45         text = movie.title,
46         style =
47             MaterialTheme.typography.headlineMedium,
48         fontWeight = FontWeight.Bold
49     )
50     Spacer(modifier = Modifier.height(8.dp))
51     Row(verticalAlignment = Alignment.CenterVertically)
52     {
53         Icon(
54             imageVector = Icons.Filled.Star,
55             contentDescription = "Rating",
56             tint = Color(0xFFFDD835),
57             modifier = Modifier.size(24.dp)
58         )
59         Spacer(modifier = Modifier.width(4.dp))
60         Text(
61             text = "${String.format("%.1f",
62 movie.voteAverage)}/10",
63             style =
64                 MaterialTheme.typography.titleMedium
65         )
66         Spacer(modifier = Modifier.height(8.dp))
67
68         movie.releaseDate?.let {
69             Text(text = "Release Date: $it", style =
70                 MaterialTheme.typography.bodyMedium)
71             Spacer(modifier = Modifier.height(8.dp))
72         }
73         Text(
74             text = movie.overview,
75             style = MaterialTheme.typography.bodyLarge
76         )
77     }

```

Presentation/ui/screens/list/MovieListScreen.kt

Table 19. Source Code presentation/ui/screens/list/MovieListScreen.kt

1	package com.example.movielist.presentation.ui.screens.list
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.foundation.lazy.LazyColumn
7	import androidx.compose.foundation.lazy.grid.GridCells
8	import
	androidx.compose.foundation.lazy.grid.LazyVerticalGrid
9	import androidx.compose.foundation.lazy.grid.items
10	import androidx.compose.foundation.lazy.items
11	import androidx.compose.material.icons.Icons
12	import androidx.compose.runtime.rememberCoroutineScope
13	import
	androidx.compose.material.icons.automirrored.filled.List
14	import androidx.compose.material.icons.filled.CheckCircle
15	import androidx.compose.material.icons.filled.Menu
16	import androidx.compose.material3.*
17	import androidx.compose.runtime.Composable
18	import androidx.compose.runtime.collectAsState
19	import androidx.compose.runtime.getValue
20	import androidx.compose.ui.Alignment
21	import androidx.compose.ui.Modifier
22	import androidx.compose.ui.platform.LocalContext
23	import androidx.compose.ui.unit.dp
24	import androidx.navigation.NavController
25	import
	com.example.movielist.presentation.ui.component.MovieCard
26	import com.example.movielist.util.LayoutType
27	import com.example.movielist.util.SettingsManager
28	import com.google.gson.Gson
29	import kotlinx.coroutines.launch
30	
31	@OptIn(ExperimentalMaterial3Api::class)
32	@Composable
33	fun MovieListScreen(
35	viewModel: MovieListViewModel,
36	settingsManager: SettingsManager,
37	navController: NavController
38	) {
39	val uiState by viewModel.uiState.collectAsState()
40	val layoutType by
	settingsManager.layoutType.collectAsState(initial =
	LayoutType.LIST)
41	val isDarkMode by
	settingsManager.isDarkMode.collectAsState(initial = false)
42	val context = LocalContext.current
43	val scope = rememberCoroutineScope()

```

44 Scaffold(
45     topBar = {
46         TopAppBar(
47             title = { Text("Popular Movies") },
48             actions = {
49                 IconButton(onClick = {
50                     scope.launch {
51 settingsManager.toggleDarkMode()
52                     }
53                 }) {
54                     Icon(
55                         imageVector = if (isDarkMode)
56 Icons.Filled.CheckCircle else Icons.Filled.CheckCircle,
57                         contentDescription = "Toggle
58 Dark/Light Mode"
59                     )
60                 }
61             }
62             IconButton(onClick = {
63                 scope.launch {
64                     val newLayout = if (layoutType
65 == LayoutType.LIST) LayoutType.GRID else LayoutType.LIST
66 settingsManager.setLayoutType(newLayout)
67                     }
68                 }) {
69                     Icon(
70                         imageVector = if (layoutType ==
71 LayoutType.LIST) Icons.Filled.Menu else
72 Icons.AutoMirrored.Filled.List,
73                         contentDescription = "Change
74 Layout"
75                     )
76                 }
77             }
78         ) { paddingValues ->
79             Box(
80                 modifier = Modifier
81                     .fillMaxSize()
82                     .padding(paddingValues)
83             ) {
84                 if (uiState.isLoading &&
85 uiState.movies.isEmpty()) {
86                     CircularProgressIndicator(modifier =
87 Modifier.align(Alignment.Center))
88                 } else if (uiState.errorMessage != null &&
89 uiState.movies.isEmpty()) {
90                     Text(
91                         text = uiState.errorMessage ?: "Unknown

```

```
86         color =  
MaterialTheme.colorScheme.error,  
87         modifier =  
Modifier.align(Alignment.Center)  
88     )  
89     } else {  
90         if (layoutType == LayoutType.LIST) {  
91             LazyColumn(  
92                 modifier = Modifier.fillMaxSize(),  
93                 contentPadding =  
94                 PaddingValues(16.dp),  
95                 verticalArrangement =  
96                 Arrangement.spacedBy(16.dp)  
97             ) {  
98                 items(uiState.movies) { movie ->  
99                     MovieCard(  
100                         movie = movie,  
101                         modifier =  
102                         Modifier.fillMaxWidth(),  
103                         onClick = {  
selectedMovie -> val movieJson =  
Uri.encode(Gson().toJson(selectedMovie))  
  
navController.navigate("movie_detail/$movieJson")  
},  
104                         onClick = { tmdbUrl ->  
105                             val intent =  
106                             Intent(Intent.ACTION_VIEW, Uri.parse(tmdbUrl))  
107                             context.startActivity(intent)  
108                             }  
109                         })  
110                     }  
111                 } else {  
112                     LazyVerticalGrid(  
113                         columns = GridCells.Fixed(2),  
114                         modifier = Modifier.fillMaxSize(),  
115                         contentPadding =  
116                         PaddingValues(16.dp),  
117                         verticalArrangement =  
118                         Arrangement.spacedBy(16.dp),  
119                         horizontalArrangement =  
120                         Arrangement.spacedBy(16.dp)  
121                     ) {  
items(uiState.movies) { movie ->  
MovieCard(  
movie = movie,  
onClick = {  
selectedMovie ->
```

121	val movieJson = Uri.encode(Gson().toJson(selectedMovie))
122	navController.navigate("movie_detail/\$movieJson")
123	},
124	onTmdbClick = { tmdbUrl ->
125	val intent = Intent(Intent.ACTION_VIEW, Uri.parse(tmdbUrl))
126	context.startActivity(intent)
127	}
128	)
129	}
130	}
131	}
132	}
133	}
134	}
135	}

Presentation/ui/screens/list/MovieListViewModel.kt

Table 20. Source Code presentation/ui/screens/list/MovieListViewModel.kt

1	package com.example.movieslist.presentation.ui.screens.list
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.movieslist.core.common.NetworkResult
6	import com.example.movieslist.domain.model.Movie
7	import com.example.movieslist.domain.usecase.GetMoviesUseCase
8	import kotlinx.coroutines.flow.MutableStateFlow
9	import kotlinx.coroutines.flow.StateFlow
10	import kotlinx.coroutines.flow.asStateFlow
11	import kotlinx.coroutines.flow.catch
12	import kotlinx.coroutines.flow.onStart
13	import kotlinx.coroutines.launch
14	
15	data class MovieListUiState(16 val movies: List<Movie> = emptyList(), 17 val isLoading: Boolean = false, 18 val errorMessage: String? = null 19)
20	
21	class MovieListViewModel(22 private val getMoviesUseCase: GetMoviesUseCase 23) : ViewModel() { 24 25 private val _uiState =


```

MutableStateFlow(MovieListUiState())
26     val uiState: StateFlow<MovieListUiState> =
    _uiState.asStateFlow()
27
28     init {
29         observeMovies()
30         refreshData()
31     }
32
33     private fun observeMovies() {
34         viewModelScope.launch {
35             getMoviesUseCase.getMoviesFromDb()
36                 .onStart { _uiState.value =
    _uiState.value.copy(isLoading = true) }
37                 .catch { e ->
38                     _uiState.value = _uiState.value.copy(
39                         isLoading = false,
40                         errorMessage = "Gagal memuat data
dari database: ${e.message}"
41                     )
42                 }
43                 .collect { movies ->
44                     _uiState.value = _uiState.value.copy(
45                         isLoading = false,
46                         movies = movies
47                     )
48                 }
49         }
50     }
51
52     fun refreshData() {
53         viewModelScope.launch {
54             _uiState.value = _uiState.value.copy(isLoading
= true, errorMessage = null)
55             when (val result =
getMoviesUseCase.refreshMoviesFromApi()) {
56                 is NetworkResult.Success -> {
57                     _uiState.value =
    _uiState.value.copy(isLoading = false)
58                 }
59                 is NetworkResult.Error -> {
60                     _uiState.value = _uiState.value.copy(
61                         isLoading = false,
62                         errorMessage = result.message
63                     )
64                 }
65             } else -> {}
66         }
67     }
68 }
69

```

Presentation/ui/screens/list/MovieListViewModelFactory.kt

Table 21. Source Code presentation/ui/screens/list/MovieListViewModelFactory.kt

1	package com.example.movielist.presentation.ui.screens.list
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
6	com.example.movielist.domain.usecase.GetMoviesUseCase
7	
8	class MovieListViewModelFactory(9 private val getMoviesUseCase: GetMoviesUseCase 10) : ViewModelProvider.Factory { 11 12 override fun <T : ViewModel> create(modelClass: 13 Class<T>): T { 14 if 15 (modelClass.isAssignableFrom(MovieListViewModel::class.java 16)) { 17 @Suppress("UNCHECKED_CAST") 18 return MovieListViewModel(getMoviesUseCase) as 19 T 20 } 21 throw IllegalArgumentException("Unknown ViewModel 22 class") 23 } 24 }

B. Output Program



Warrior Queen

★ 3.4/10

Release Date: 1987-01-26

In ancient Pompeii, slaves are bought and sold for household chores and sex. A mysterious queen moves among the elite, meantime secretly helping the slaves to escape.

Eventually her life is also in jeopardy, and as the volcano erupts she and the slaves attempt to escape while being chased by the military.

Gambar 1. Tampilan Layar Detail

Popular Movies



Warrior Queen

Rating: 3.4/10

[TMDB](#)[Detail](#)

Thor: Ragnarok

Rating: 7.6/10

[TMDB](#)[Detail](#)

The Accountant

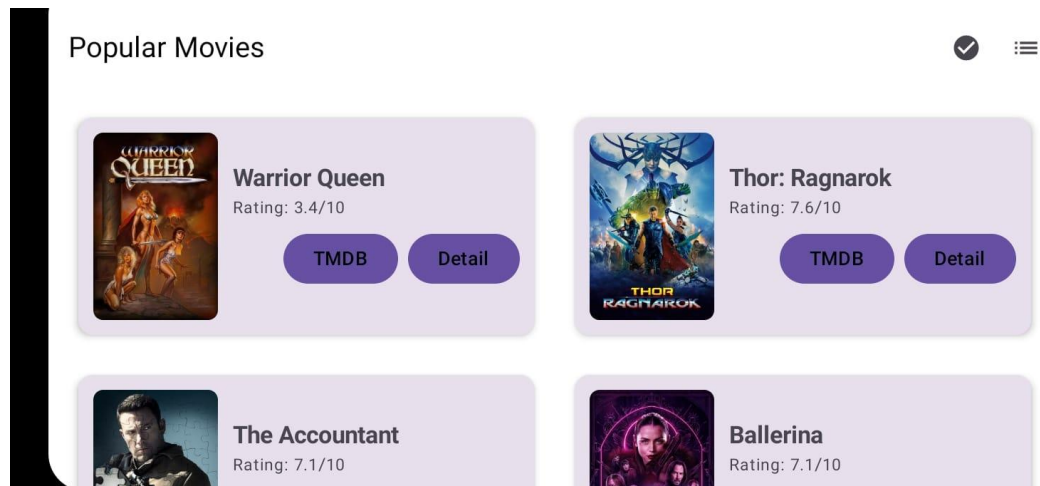
Rating: 7.1/10

[TMDB](#)[Detail](#)

Ballerina

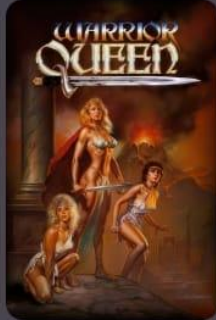
Rating: 7.1/10

Gambar 2. Tampilan Awal



Gambar 3. Tampilan Awal Landscape

Popular Movies



Warrior Queen

Rating: 3.4/10

[TMDB](#)

[Detail](#)

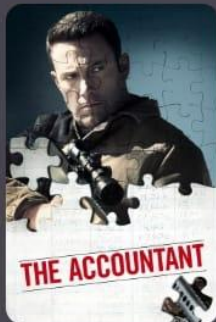


Thor: Ragnarok

Rating: 7.6/10

[TMDB](#)

[Detail](#)



The Accountant

Rating: 7.1/10

[TMDB](#)

[Detail](#)



Ballerina

Rating: 7.1/10



C. Pembahasan

Presentation/MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–27], diimport berbagai komponen yang diperlukan untuk aktivitas dan tampilan aplikasi. `ComponentActivity` diimport untuk membuat activity berbasis komponen, yang digunakan untuk menginisialisasi dan mengatur UI dalam aplikasi Android menggunakan Jetpack Compose. `setContent` digunakan untuk menetapkan tampilan UI aplikasi. `isSystemInDarkTheme` digunakan untuk mendeteksi apakah tema gelap sistem sedang aktif. Selain itu, berbagai fungsi dan class seperti `Surface`, `Text`, `MaterialTheme`, `Modifier`, `ViewModel`, `collectAsState`, `NavHost`, `composable`, `rememberNavController`, `navArgument`, dan `NavType` diimport untuk membangun dan mengelola UI serta navigasi menggunakan Jetpack Compose. `AppContainer` digunakan untuk mengelola dependensi aplikasi secara global. `Movie`, `MovieDetailScreen`, `MovieListScreen`, `MovieListViewModel`, dan `MovieListTheme` diimport sebagai entitas dan composable utama aplikasi yang menampilkan daftar dan detail film, serta tema aplikasi. `Gson` diimport untuk keperluan serialisasi dan deserialisasi objek JSON.

Pada baris [28], `MainActivity` dideklarasikan sebagai turunan dari `ComponentActivity`. Class ini merupakan titik awal aplikasi yang dijalankan ketika aplikasi dibuka. Sebagai aktivitas utama, `MainActivity` bertanggung jawab untuk mengatur siklus hidup aplikasi dan menampilkan antarmuka pengguna menggunakan Jetpack Compose.

Pada baris [29], dalam method `onCreate()`, kita memanggil `super.onCreate(savedInstanceState)` untuk memastikan siklus hidup activity dijalankan sesuai dengan standar.

Pada baris [32], variabel `appContainer` dideklarasikan dan diinisialisasi dengan membuat instance `AppContainer` yang memerlukan `applicationContext`. `AppContainer` ini akan berfungsi sebagai wadah untuk semua dependensi aplikasi, seperti repositori, use case, dan view model factory, yang kemudian akan diakses di seluruh bagian aplikasi.

Pada baris [34–72], `setContent` digunakan untuk menetapkan tampilan UI aplikasi menggunakan Jetpack Compose.

Pada baris [35], variabel `settingsManager` dideklarasikan dan diinisialisasi dengan mengakses `appContainer.settingsManager`, yang menyediakan fungsionalitas untuk mengelola pengaturan aplikasi, seperti tema dan jenis tata letak.

Pada baris [36], variabel `isSystemDark` dideklarasikan dan diinisialisasi dengan memanggil `isSystemInDarkTheme()`, yang mengembalikan `true` jika tema gelap aktif di sistem operasi perangkat.

Pada baris [37], variabel `isDarkMode` dideklarasikan dan diobservasi dari `settingsManager.isDarkMode.collectAsState()`, dengan nilai awal yang berasal dari `isSystemDark`. Variabel ini akan secara reaktif memperbarui UI ketika mode gelap diubah melalui pengaturan aplikasi atau sistem.

Pada baris [39], `MovieListTheme` digunakan sebagai tema utama aplikasi, dengan properti `darkTheme` diatur berdasarkan nilai `isDarkMode` yang diobservasi, memastikan konsistensi gaya visual aplikasi sesuai mode gelap atau terang.

Pada baris [40–67], `Surface` dideklarasikan sebagai komponen pembungkus untuk menampilkan konten UI. `modifier = Modifier.fillMaxSize()` membuat `Surface` mengambil seluruh ruang yang tersedia di layar, sementara `color = MaterialTheme.colorScheme.background` menetapkan warna latar belakang sesuai skema warna tema aplikasi.

Pada baris [44], variabel `navController` dideklarasikan dan diinisialisasi dengan `rememberNavController()`, yang menyediakan kontrol untuk navigasi antar layar dalam aplikasi. `navController` digunakan untuk mengelola transisi antar tampilan atau komponen dalam aplikasi, seperti berpindah dari tampilan daftar film ke tampilan detail film.

Pada baris [45–47], variabel `movieListViewModel` dideklarasikan dan diinisialisasi menggunakan fungsi `viewModel()`, yang mengambil instance dari `MovieListViewModel` dengan factory yang disediakan oleh `appContainer.movieListViewModelFactory`. `ViewModel` ini digunakan untuk menyimpan dan mengelola data serta logika bisnis yang diperlukan oleh UI daftar film.

Pada baris [49–65], `NavHost` dideklarasikan untuk mengelola navigasi antar tampilan. `startDestination` ditetapkan ke `"movie_list"`, yang berarti aplikasi akan memulai dengan menampilkan tampilan daftar film. Di dalam `NavHost`, ada dua rute yang didefinisikan.

Pada baris [50–56], rute "movie_list" akan memanggil MovieListScreen, dengan viewModel dan settingsManager yang relevan, serta navController untuk navigasi.

Pada baris [57–65], rute "movie_detail/{movieJson}" didefinisikan untuk menampilkan detail film. Rute ini mengambil argumen movieJson dari URL. movieJson kemudian dideserialisasi menjadi objek Movie menggunakan Gson. Jika objek Movie tidak null, MovieDetailScreen akan dipanggil untuk menampilkan detail film tersebut. Jika data film tidak ditemukan, maka akan ditampilkan pesan "Error: Movie details not available." menggunakan Text dengan warna error dari tema.

AppContainer.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–11], diimport berbagai komponen yang diperlukan untuk manajemen dependensi aplikasi. Context diimport untuk menyediakan akses ke sumber daya dan layanan sistem. AppDatabase diimport untuk berinteraksi dengan database lokal. KtorApiClient diimport untuk menangani komunikasi dengan API remote. MovieRepositoryImpl dan MovieRepository diimport untuk mengimplementasikan dan mendefinisikan antarmuka repositori data film. GetMoviesUseCase diimport sebagai use case untuk mendapatkan data film. MovieListViewModelFactory diimport sebagai factory untuk membuat instance MovieListViewModel. SettingsManager diimport untuk mengelola pengaturan aplikasi.

Pada baris [12], AppContainer dideklarasikan sebagai sebuah kelas yang akan bertanggung jawab untuk menyediakan semua dependensi yang dibutuhkan oleh aplikasi, mengimplementasikan pola Dependency Injection. Konstruktornya menerima sebuah Context sebagai parameter, yang dibutuhkan untuk inisialisasi beberapa dependensi.

Pada baris [14–16], variabel appDatabase dideklarasikan menggunakan by lazy untuk memastikan inisialisasi yang malas (lazy initialization). AppDatabase.create(context) dipanggil untuk membuat instance database Room, yang digunakan untuk menyimpan data film secara lokal.

Pada baris [18–20], variabel movieDao dideklarasikan menggunakan by lazy. Ini mengakses Movie Data Access Object (DAO) dari appDatabase, yang menyediakan metode untuk berinteraksi dengan tabel film di database.

Pada baris [22–24], variabel `ktorApiClient` dideklarasikan menggunakan `by lazy`. Ini mengacu langsung pada objek `KtorApiClient`, yang bertanggung jawab untuk melakukan panggilan jaringan ke API TheMovieDB.

Pada baris [26–28], variabel `movieRepository` dideklarasikan menggunakan `by lazy`. Ini membuat instance `MovieRepositoryImpl`, yang merupakan implementasi dari `MovieRepository`. Repositori ini bertugas mengelola sumber data, baik dari API (`ktorApiClient`) maupun dari database lokal (`movieDao`).

Pada baris [30–32], variabel `getMoviesUseCase` dideklarasikan menggunakan `by lazy`. Ini membuat instance `GetMoviesUseCase`, yang menerima `movieRepository` sebagai dependensinya. Use case ini mendefinisikan logika bisnis untuk mendapatkan daftar film.

Pada baris [34–36], variabel `movieListViewModelFactory` dideklarasikan menggunakan `by lazy`. Ini membuat instance `MovieListViewModelFactory`, yang menerima `getMoviesUseCase` sebagai dependensinya. Factory ini digunakan oleh Jetpack Compose untuk membuat instance `MovieListViewModel` dengan dependensi yang diperlukan.

Pada baris [38–40], variabel `settingsManager` dideklarasikan menggunakan `by lazy`. Ini membuat instance `SettingsManager`, yang menerima `context` sebagai dependensinya. `SettingsManager` bertanggung jawab untuk mengelola preferensi pengguna seperti tema aplikasi dan jenis tata letak (`grid/list`).

MyApplication.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], diimport `Application`, sebuah kelas dasar yang menyediakan fungsionalitas inti untuk mengelola keadaan aplikasi secara global dan siklus hidupnya.

Pada baris [5], `MyApplication` dideklarasikan sebagai sebuah kelas yang merupakan turunan dari `Application`. Class ini berfungsi sebagai titik masuk khusus aplikasi saat pertama kali diluncurkan, memungkinkan inisialisasi komponen yang perlu ada sepanjang siklus hidup aplikasi.

Pada baris [6], variabel `container` dideklarasikan dengan kata kunci `lateinit`. Ini berarti `container` adalah sebuah instance dari `AppContainer` yang akan diinisialisasi nanti, memastikan bahwa objek `AppContainer` akan tersedia dan dapat diakses di seluruh bagian aplikasi.

Pada baris [8–11], dalam method `onCreate()`, `super.onCreate()` dipanggil untuk menjalankan inisialisasi standar dari kelas `Application` induk. Kemudian, pada baris [10], container diinisialisasi dengan membuat instance baru dari `AppContainer`, yang menerima `this` (instance `MyApplication` itu sendiri sebagai `Context`) sebagai argumen. Ini memastikan bahwa `AppContainer` dibuat satu kali saat aplikasi dimulai dan dapat diakses sebagai singleton oleh komponen lain di seluruh aplikasi.

Core/common/NetworkResult.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], dideklarasikan kelas `NetworkResult` sebagai sealed class. Sealed class digunakan untuk merepresentasikan hierarki kelas terbatas, di mana semua subclass diketahui pada waktu kompilasi. Ini sangat berguna untuk merepresentasikan status yang berbeda (misalnya, sukses, error, loading) dari suatu operasi jaringan. Kelas ini memiliki dua properti opsional: `data` (bertipe `T`, yang bisa null) untuk menyimpan hasil sukses, dan `message` (bertipe `String`, yang bisa null) untuk menyimpan pesan error.

Pada baris [7], dideklarasikan `Success` sebagai subclass dari `NetworkResult`. Kelas `Success` merepresentasikan keadaan ketika operasi jaringan berhasil. Ia memiliki satu parameter `data` bertipe `T`, yang harus berisi hasil operasi yang sukses.

Pada baris [8], dideklarasikan `Error` sebagai subclass dari `NetworkResult`. Kelas `Error` merepresentasikan keadaan ketika operasi jaringan gagal. Ia memiliki dua parameter: `message` bertipe `String` yang wajib untuk menjelaskan error, dan `data` opsional bertipe `T` yang bisa null (berguna jika ada data parsial atau cache yang bisa ditampilkan meskipun ada error).

Pada baris [9], dideklarasikan `Loading` sebagai subclass dari `NetworkResult`. Kelas `Loading` merepresentasikan keadaan ketika operasi jaringan sedang berlangsung. Kelas ini tidak memiliki properti karena pada tahap ini, data atau pesan error belum tersedia.

Data/local/dao/MovieDao.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–9], diimport berbagai komponen yang diperlukan dari pustaka `Room` dan Kotlin Coroutines. `@Dao` diimport untuk menandai antarmuka sebagai `Data Access Object` `Room`, yang bertanggung jawab untuk mendefinisikan metode interaksi database. `@Insert`,

@OnConflictStrategy, @Query, dan @Transaction adalah anotasi Room yang digunakan untuk menentukan jenis operasi database. MovieEntity diimport sebagai representasi objek film di database. Flow diimport dari kotlinx.coroutines.flow untuk mendukung pengumpulan data asinkron.

Pada baris [11], antarmuka MovieDao dideklarasikan dan dianotasi dengan @Dao, menandakan bahwa ini adalah Data Access Object yang digunakan oleh Room untuk berinteraksi dengan database.

Pada baris [14–15], fungsi getMovies() dideklarasikan. Dianotasi dengan @Query("SELECT * FROM movies"), fungsi ini akan mengeksekusi kueri SQL untuk mengambil semua baris dari tabel bernama "movies". Fungsi ini mengembalikan Flow<List<MovieEntity>>, yang berarti ia akan secara asinkron memancarkan daftar MovieEntity setiap kali ada perubahan pada data film di database.

Pada baris [17–18], fungsi insertMovies() dideklarasikan sebagai suspend fungsi. Dianotasi dengan @Insert(onConflict = OnConflictStrategy.REPLACE), fungsi ini akan menyisipkan daftar objek MovieEntity ke dalam tabel "movies". Jika ada konflik (misalnya, entitas dengan primary key yang sama sudah ada), strategi OnConflictStrategy.REPLACE akan mengganti entitas yang sudah ada dengan yang baru.

Pada baris [20–21], fungsi clearMovies() dideklarasikan sebagai suspend fungsi. Dianotasi dengan @Query("DELETE FROM movies"), fungsi ini akan mengeksekusi kueri SQL untuk menghapus semua baris dari tabel "movies", secara efektif mengosongkan tabel.

Pada baris [23–27], fungsi clearAndInsertMovies() dideklarasikan sebagai suspend fungsi. Dianotasi dengan @Transaction, fungsi ini akan menjalankan operasi clearMovies() dan insertMovies() dalam satu transaksi database tunggal. Ini memastikan bahwa kedua operasi berhasil atau keduanya gagal, menjaga integritas data. Secara spesifik, ia akan menghapus semua film yang ada, lalu menyisipkan daftar film yang baru.

Data/local/model/MovieEntity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport komponen yang diperlukan dari pustaka Room. @Entity diimport untuk menandai kelas sebagai entitas database Room, yang berarti kelas ini akan dipetakan

ke sebuah tabel dalam database. `@PrimaryKey` diimport untuk menandai sebuah properti sebagai kunci utama tabel.

Pada baris [7], kelas data `MovieEntity` dideklarasikan. Kelas ini dianotasi dengan `@Entity(tableName = "movies")`, yang berarti Room akan membuat tabel dalam database dengan nama "movies" yang strukturnya sesuai dengan properti kelas ini.

Pada baris [8], properti `id` dideklarasikan sebagai `Int` dan dianotasi dengan `@PrimaryKey`. Ini menetapkan `id` sebagai kunci utama tabel "movies", memastikan setiap entri film memiliki identifikasi unik dan tidak ada duplikasi berdasarkan `id`.

Pada baris [9], properti `title` dideklarasikan sebagai `String`. Ini akan menyimpan judul film. Catatan `NON-NULLABLE` menunjukkan bahwa kolom ini tidak boleh berisi nilai `null` di database.

Pada baris [10], properti `overview` dideklarasikan sebagai `String`. Ini akan menyimpan ringkasan atau deskripsi singkat film. Catatan `NON-NULLABLE` juga menunjukkan bahwa kolom ini tidak boleh berisi nilai `null` di database.

Pada baris [11], properti `posterPath` dideklarasikan sebagai `String`. Ini akan menyimpan jalur (path) relatif ke gambar poster film.

Pada baris [12], properti `backdropPath` dideklarasikan sebagai `String`. Ini akan menyimpan jalur (path) relatif ke gambar backdrop film, yang biasanya digunakan sebagai latar belakang.

Pada baris [13], properti `releaseDate` dideklarasikan sebagai `String`. Ini akan menyimpan tanggal rilis film.

Pada baris [14], properti `voteAverage` dideklarasikan sebagai `Double`. Ini akan menyimpan rata-rata rating suara atau nilai voting film.

Pada baris [15], properti `popularity` dideklarasikan sebagai `Double`. Ini akan menyimpan nilai popularitas film.

Data/local/AppDatabase.kt:

Pada baris [1], dideklarasikan package tempat file ini berada, yaitu `com.example.booklist.screens`.

Pada baris [3–8], diimport berbagai komponen yang diperlukan dari pustaka Room dan Android, yaitu `Context`, `@Database`, `Room`, `RoomDatabase`, `MovieDao`, dan `MovieEntity`.

Pada baris [10], kelas abstrak AppDatabase dideklarasikan sebagai turunan dari RoomDatabase. Kelas ini dianotasi dengan @Database(entities = [MovieEntity::class], version = 1, exportSchema = false). entities = [MovieEntity::class] menunjukkan MovieEntity adalah satu-satunya entitas yang akan disertakan dalam database ini. version = 1 menetapkan versi database. exportSchema = false berarti Room tidak akan mengeksport skema database ke dalam folder.

Pada baris [12], fungsi abstrak movieDao() dideklarasikan. Fungsi ini akan diimplementasikan secara otomatis oleh Room untuk menyediakan instance dari MovieDao, yang memungkinkan interaksi dengan tabel film.

Pada baris [14], blok companion object dideklarasikan. Ini adalah cara di Kotlin untuk mendefinisikan anggota statis yang terkait dengan kelas itu sendiri, bukan dengan instance kelas.

Pada baris [16], variabel INSTANCE dideklarasikan dengan anotasi @Volatile dan diinisialisasi sebagai null. @Volatile memastikan bahwa nilai variabel ini selalu up-to-date dan terlihat oleh semua thread, yang penting untuk mencegah masalah konkurensi saat membuat instance database tunggal.

Pada baris [18–29], fungsi create() dideklarasikan. Fungsi ini adalah metode singleton yang digunakan untuk mendapatkan atau membuat instance tunggal dari AppDatabase. return INSTANCE ?: synchronized(this) { ... } akan mengembalikan INSTANCE yang sudah ada jika tidak null. Jika INSTANCE adalah null, maka blok synchronized(this) akan dieksekusi. synchronized(this) memastikan bahwa hanya satu thread yang dapat masuk ke blok ini pada satu waktu, mencegah pembuatan beberapa instance database dalam lingkungan multi-thread. Di dalam blok synchronized, Room.databaseBuilder() digunakan untuk membangun instance database, menggunakan context.applicationContext sebagai konteks aplikasi, AppDatabase::class.java sebagai kelas database, dan "movie_database" sebagai nama file database. .fallbackToDestructiveMigration() ditambahkan ke builder, yang merupakan strategi migrasi database yang akan menghapus semua data yang ada dan membuat ulang database jika terjadi perubahan skema dan tidak ada aturan migrasi eksplisit yang ditentukan. Ini sangat berguna selama fase pengembangan untuk menghindari crash karena perubahan skema. .build() menyelesaikan proses pembangunan database. INSTANCE = instance

menyimpan instance database yang baru dibuat ke variabel INSTANCE, dan instance mengembalikan instance database tersebut.

Data/mapper/MovieMapper.kt:

Pada baris [1], ditentukan package tempat file ini berada, yaitu com.example.booklist.

Pada baris [3–5], diimport berbagai kelas model yang diperlukan untuk pemetaan data. MovieEntity diimport sebagai representasi data film di database lokal. MovieResultDto diimport sebagai representasi data film yang diterima dari API. Movie diimport sebagai representasi model domain film, yang merupakan objek data yang digunakan di lapisan domain aplikasi.

Pada baris [7–16], fungsi ekstensi toEntity() dideklarasikan untuk kelas MovieResultDto. Fungsi ini bertanggung jawab untuk mengubah objek MovieResultDto (data dari API) menjadi objek MovieEntity (data untuk database lokal). Di dalamnya, setiap properti dari MovieResultDto dipetakan ke properti yang sesuai di MovieEntity. Operator ?: (Elvis operator) atau .orEmpty() digunakan untuk memberikan nilai default (0, 0.0, atau string kosong) jika properti dari MovieResultDto bersifat nullable, memastikan bahwa MovieEntity yang dihasilkan memiliki nilai yang valid untuk semua properti yang dideklarasikan sebagai non-nullable.

Pada baris [20–28], fungsi ekstensi toDomain() dideklarasikan untuk kelas MovieEntity. Fungsi ini bertanggung jawab untuk mengubah objek MovieEntity (data dari database lokal) menjadi objek Movie (model domain). Proses ini memetakan properti dari MovieEntity ke properti yang sesuai di Movie, menjadikannya siap untuk digunakan di lapisan domain aplikasi, yang merupakan representasi agnostik terhadap sumber data.

Data/remote/api/ApiConstants.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], dideklarasikan object ApiConstants. Penggunaan object di Kotlin berarti bahwa ApiConstants adalah singleton, yang berarti hanya ada satu instance dari kelas ini sepanjang siklus hidup aplikasi. Ini sangat cocok untuk menyimpan konstanta global seperti URL dasar API dan kunci API.

Pada baris [4], properti `BASE_URL` dideklarasikan sebagai `const val` dengan nilai `"https://api.themoviedb.org/3/"`. `const val` berarti ini adalah konstanta waktu kompilasi, yang optimal untuk nilai-nilai yang tidak akan berubah. `BASE_URL` ini adalah alamat dasar dari API The Movie Database yang akan digunakan untuk semua permintaan API.

Pada baris [5], properti `API_KEY` dideklarasikan sebagai `const val`. Ini adalah kunci API yang diperlukan untuk mengautentikasi permintaan ke The Movie Database API. Kunci ini harus disertakan dalam setiap permintaan untuk mendapatkan akses ke data API.

Data/remote/api/KtorApiClient.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–12], diimport berbagai komponen yang diperlukan dari pustaka Ktor, Kotlinx Serialization, dan model data. `MovieResponseDto` diimport sebagai model data untuk respons API film. `HttpClient`, `call.*`, `Android`, `contentnegotiation.*`, `logging.*`, dan `request.*` diimport dari Ktor Client untuk membangun dan mengkonfigurasi klien HTTP. `json` dari `io.ktor.serialization.kotlinx.json` dan `Json` dari `kotlinx.serialization.json` diimport untuk menangani serialisasi dan deserialisasi JSON.

Pada baris [14], dideklarasikan object `KtorApiClient`. Penggunaan object di Kotlin berarti `KtorApiClient` adalah singleton, yang berarti hanya ada satu instance dari klien HTTP ini sepanjang siklus hidup aplikasi. Ini memastikan bahwa konfigurasi klien dan koneksi jaringan digunakan secara efisien.

Pada baris [15–24], properti `client` dideklarasikan dan diinisialisasi sebagai `HttpClient` yang menggunakan Android engine. Di dalamnya, beberapa plugin Ktor diinstal:

Pada baris [16–18], `install(Logging)` diinstal dengan `level = LogLevel.ALL`. Plugin ini mengaktifkan logging lengkap untuk semua permintaan dan respons HTTP, yang sangat berguna untuk debugging karena menampilkan detail seperti URL, header, dan body permintaan/respons.

Pada baris [19–24], `install(ContentNegotiation)` diinstal, dengan konfigurasi `json` yang menggunakan objek `Json` dari `kotlinx.serialization.json`. Konfigurasi JSON ini disetel dengan `ignoreUnknownKeys = true` (mengabaikan kunci JSON yang tidak ada di model data, mencegah crash jika API mengembalikan bidang baru), `isLenient = true` (memungkinkan

format JSON yang sedikit kurang ketat), dan `prettyPrint = true` (memformat output JSON agar mudah dibaca di log).

Pada baris [26–30], fungsi `suspend fun getPopularMovies(): MovieResponseDto` dideklarasikan. Fungsi ini adalah fungsi suspending, yang berarti ia dapat dipanggil dari dalam coroutine dan akan menjeda eksekusi tanpa memblokir thread hingga hasilnya tersedia. Fungsi ini bertanggung jawab untuk melakukan permintaan HTTP untuk mendapatkan daftar film populer.

Pada baris [27], `client.get(ApiConstants.BASE_URL + "movie/popular")` membuat permintaan GET ke URL film populer menggunakan `BASE_URL` dari `ApiConstants`.

Pada baris [28], `parameter("api_key", ApiConstants.API_KEY)` menambahkan parameter kueri `api_key` ke permintaan dengan nilai dari `ApiConstants.API_KEY` untuk autentikasi.

Pada baris [29], `.body()` digunakan untuk deserialisasi respons HTTP menjadi objek `MovieResponseDto`, yang kemudian dikembalikan oleh fungsi ini.

Data/remote/model/MovieDto.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–5], diimport komponen yang diperlukan dari pustaka Kotlinx Serialization. `SerializedName` diimport untuk menentukan nama bidang JSON yang berbeda dari nama properti Kotlin. `Serializable` diimport untuk menandai kelas agar dapat di-serialisasi dan de-serialisasi secara otomatis oleh `kotlinx.serialization`. Anotasi `@SuppressLint("UnsafeOptInUsageError")` digunakan untuk menekan peringatan terkait penggunaan fitur opt-in yang mungkin belum stabil, seringkali terkait dengan fitur eksperimental dalam serialisasi.

Pada baris [9–16], kelas data `MovieResponseDto` dideklarasikan. Kelas ini dianotasi dengan `@Serializable`, memungkinkan instance kelas ini diubah menjadi format data (misalnya JSON) dan sebaliknya. Kelas ini merepresentasikan struktur respons keseluruhan dari API film.

Pada baris [10], properti `page` dideklarasikan sebagai `Int?` (nullable integer), merepresentasikan nomor halaman hasil yang saat ini diambil.

Pada baris [11], properti `results` dideklarasikan sebagai `List<MovieResultDto>?` (nullable list of `MovieResultDto`), yang berisi daftar film sebenarnya yang dikembalikan oleh API.

Pada baris [12–13], properti totalPages dideklarasikan sebagai Int? dan dianotasi dengan @SerializedName("total_pages"). Ini merepresentasikan jumlah total halaman yang tersedia dari hasil API. SerialName digunakan karena nama properti di JSON adalah "total_pages" sedangkan di Kotlin adalah "totalPages".

Pada baris [14–15], properti totalResults dideklarasikan sebagai Int? dan dianotasi dengan @SerializedName("total_results"). Ini merepresentasikan jumlah total film yang cocok dengan kueri.

Pada baris [20–33], kelas data MovieResultDto dideklarasikan. Kelas ini juga dianotasi dengan @Serializable, memungkinkan serialisasi dan de-serialisasi. Kelas ini merepresentasikan detail satu film dalam daftar hasil.

Pada baris [21], properti id dideklarasikan sebagai Int? (nullable integer), yang merupakan ID unik dari film.

Pada baris [22], properti title dideklarasikan sebagai String? (nullable string), yang merupakan judul film.

Pada baris [23], properti overview dideklarasikan sebagai String? (nullable string), yang merupakan ringkasan atau deskripsi film.

Pada baris [24–25], properti posterPath dideklarasikan sebagai String? dan dianotasi dengan @SerializedName("poster_path"), yang berisi jalur (path) relatif ke gambar poster film.

Pada baris [26–27], properti releaseDate dideklarasikan sebagai String? dan dianotasi dengan @SerializedName("release_date"), yang berisi tanggal rilis film.

Pada baris [28–29], properti voteAverage dideklarasikan sebagai Double? dan dianotasi dengan @SerializedName("vote_average"), yang berisi rata-rata voting atau rating film.

Pada baris [30–31], properti backdropPath dideklarasikan sebagai String? dan dianotasi dengan @SerializedName("backdrop_path"), yang berisi jalur (path) relatif ke gambar backdrop film.

Pada baris [32], properti popularity dideklarasikan sebagai Double? (nullable double), yang berisi nilai popularitas film.

Data/repository/MovieRepositoryImpl.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–12], diimport berbagai komponen yang diperlukan untuk implementasi repositori. `NetworkResult` diimport untuk merepresentasikan status operasi jaringan. `MovieDao` diimport untuk berinteraksi dengan database lokal. `toDomain` dan `toEntity` diimport sebagai fungsi mapper untuk mengubah data antar lapisan. `KtorApiClient` diimport untuk melakukan panggilan API. `MovieResponseDto` diimport sebagai model data respons dari API. `Movie` diimport sebagai model domain film. `MovieRepository` diimport sebagai antarmuka repositori yang diimplementasikan. `Flow` dan `map` dari `kotlinx.coroutines.flow` diimport untuk menangani aliran data asinkron. `IOException` diimport untuk menangani pengecualian terkait input/output.

Pada baris [14], kelas `MovieRepositoryImpl` dideklarasikan. Kelas ini mengimplementasikan antarmuka `MovieRepository` dan bertanggung jawab untuk mengelola data film dari berbagai sumber (API dan database lokal). Konstruktornya menerima `apiClient` (instance `KtorApiClient`) dan `movieDao` (instance `MovieDao`) sebagai dependensinya.

Pada baris [19–23], fungsi `getMovies()` di-override dari antarmuka `MovieRepository`. Fungsi ini mengembalikan `Flow<List<Movie>>`. Di dalamnya, `movieDao.getMovies()` dipanggil untuk mendapatkan aliran data `List<MovieEntity>` dari database lokal. Kemudian, `.map { entities -> entities.map { it.toDomain() } }` digunakan untuk mengubah setiap `MovieEntity` dalam daftar menjadi objek `Movie` (model domain) menggunakan fungsi `toDomain()` sebelum memancarkannya sebagai aliran data.

Pada baris [25–34], fungsi `refreshMovies()` di-override dari antarmuka `MovieRepository` sebagai suspend fungsi. Fungsi ini bertanggung jawab untuk memperbarui data film dari API dan menyimpannya ke database lokal, mengembalikan `NetworkResult<Unit>` untuk menunjukkan status operasi.

Pada baris [26], blok `try` dimulai untuk menangani potensi error selama operasi jaringan dan database.

Pada baris [27], `val response: MovieResponseDto = apiClient.getPopularMovies()` memanggil API Ktor untuk mendapatkan daftar film populer.

Pada baris [28], `val movieEntities = response.results?.map { it.toEntity() } ?: emptyList()` mengambil daftar hasil film dari respons API. Setiap `MovieResultDto` kemudian diubah menjadi `MovieEntity` menggunakan fungsi `toEntity()`. Jika `response.results` null, maka akan menghasilkan `emptyList()`.

Pada baris [29], `movieDao.clearAndInsertMovies(movieEntities)` dipanggil untuk menghapus semua film yang ada di database lokal dan kemudian menyisipkan daftar film yang baru didapatkan dari API. Operasi ini berjalan dalam satu transaksi.

Pada baris [30], jika semua operasi berhasil, `NetworkResult.Success(Unit)` dikembalikan, menunjukkan keberhasilan operasi tanpa data spesifik yang dikembalikan.

Pada baris [31–32], blok `catch (e: IOException)` menangani pengecualian `IOException`, yang biasanya terjadi karena masalah koneksi jaringan (misalnya, tidak ada internet). Dalam kasus ini, `NetworkResult.Error` dikembalikan dengan pesan error yang relevan untuk pengguna.

Pada baris [33–34], blok `catch (e: Exception)` adalah penangkap umum untuk pengecualian lain yang mungkin terjadi. Dalam kasus ini, `NetworkResult.Error` dikembalikan dengan pesan error yang berisi detail dari `e.message` untuk diagnosis lebih lanjut.

Domain/model/Movie.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], kelas data `Movie` dideklarasikan. Kelas data ini adalah representasi objek film di lapisan domain aplikasi, yang berarti ia berisi data yang relevan untuk UI tanpa terpapar detail implementasi dari sumber data (baik API maupun database).

Pada baris [4–10], dideklarasikan properti-properti yang membentuk objek `Movie`. Properti `id` dideklarasikan sebagai `Int`, yang merupakan ID unik untuk film. Properti `title` dideklarasikan sebagai `String`, yang menyimpan judul film. Properti `overview` dideklarasikan sebagai `String`, yang menyimpan ringkasan atau sinopsis film. Properti `posterPath` dideklarasikan sebagai `String?` (`nullable String`), yang menyimpan jalur relatif ke gambar poster film. Ini `nullable` karena mungkin tidak selalu tersedia. Properti `backdropPath` dideklarasikan sebagai `String?` (`nullable String`), yang menyimpan jalur relatif ke gambar latar belakang (`backdrop`) film. Ini juga `nullable`. Properti `voteAverage` dideklarasikan sebagai `Double`, yang menyimpan nilai rata-rata vote atau rating film. Properti `releaseDate` dideklarasikan sebagai `String?` (`nullable String`), yang menyimpan tanggal rilis film. Ini `nullable`.

Pada baris [11–13], fungsi `getPosterUrl()` dideklarasikan. Fungsi ini mengembalikan URL lengkap dari gambar poster film. Ia memeriksa apakah `posterPath` tidak `null` dan tidak kosong, jika ya, ia akan menggabungkan `BASE_URL` dari The Movie Database untuk

gambar (<https://image.tmbd.org/t/p/w500>) dengan posterPath yang tersedia. Jika posterPath null atau kosong, fungsi ini mengembalikan string kosong.

Pada baris [15–17], fungsi getBackdropUrl() dideklarasikan. Mirip dengan getPosterUrl(), fungsi ini mengembalikan URL lengkap dari gambar backdrop film. Ia menggunakan ukuran yang berbeda (w780) dan juga memeriksa backdropPath untuk memastikan tidak null atau kosong, mengembalikan string kosong jika tidak ada.

Pada baris [19–21], fungsi getTmdbUrl() dideklarasikan. Fungsi ini mengembalikan URL langsung ke halaman film di situs web The Movie Database, menggunakan id film untuk membuat tautan yang spesifik.

Domain/repository/MovieRepository.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–5], diimport berbagai komponen yang diperlukan. NetworkResult diimport untuk merepresentasikan status operasi jaringan. Movie diimport sebagai model domain film. Flow diimport dari kotlinx.coroutines.flow untuk mendukung pengumpulan data asinkron.

Pada baris [7], antarmuka MovieRepository dideklarasikan. Antarmuka ini mendefinisikan kontrak untuk operasi data film, memisahkan lapisan domain dari detail implementasi data.

Pada baris [8], fungsi getMovies() dideklarasikan. Fungsi ini mengembalikan Flow<List<Movie>>, yang berarti ia akan menyediakan aliran data asinkron berisi daftar objek Movie. Ini digunakan untuk mengambil data film yang akan diobservasi oleh lapisan presentasi. Pada baris [9], fungsi refreshMovies() dideklarasikan sebagai suspend fungsi. Fungsi ini bertanggung jawab untuk memperbarui data film, biasanya dengan mengambil data terbaru dari sumber eksternal (misalnya API). Ia mengembalikan NetworkResult<Unit>, yang menunjukkan status operasi (sukses, error, atau loading) tanpa mengembalikan data spesifik (hanya Unit sebagai penanda keberhasilan).

Domain/usecase/GetMoviesUseCase.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–6], diimport berbagai komponen yang diperlukan. NetworkResult diimport untuk merepresentasikan status operasi jaringan. Movie diimport sebagai model domain film.

MovieRepository diimport sebagai antarmuka repositori untuk mendapatkan data film. Flow diimport dari `kotlinx.coroutines.flow` untuk mendukung pengumpulan data asinkron.

Pada baris [8], kelas `GetMoviesUseCase` dideklarasikan. Kelas ini merupakan bagian dari lapisan domain dan mengimplementasikan logika bisnis untuk mengambil dan menyegarkan data film. Konstruktornya menerima `movieRepository` sebagai dependensinya, yang memungkinkan use case ini berinteraksi dengan lapisan data.

Pada baris [9], fungsi `getMoviesFromDb()` dideklarasikan. Fungsi ini mengembalikan `Flow<List<Movie>>`. Ia secara langsung mendelegasikan panggilan ke `movieRepository.getMovies()`, yang berarti use case ini menyediakan cara bagi lapisan presentasi untuk mendapatkan aliran data film dari database lokal.

Pada baris [11], fungsi `refreshMoviesFromApi()` dideklarasikan sebagai suspend fungsi. Fungsi ini juga mendelegasikan panggilan ke `movieRepository.refreshMovies()`. Ini adalah fungsi yang bertanggung jawab untuk memicu pembaruan data film dari API dan menyimpannya ke database, mengembalikan `NetworkResult<Unit>` untuk menunjukkan status operasi (sukses, error, atau loading).

presentation/ui/component/MovieCard.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–19], diimport berbagai komponen yang diperlukan dari Jetpack Compose dan pustaka lainnya. `androidx.compose.foundation` diimport untuk elemen UI dasar seperti tata letak dan interaksi. `androidx.compose.material3` diimport untuk komponen Material Design 3 seperti Card, Button, Text, MaterialTheme, CardDefaults, dan ExperimentalMaterial3Api. `androidx.compose.runtime` diimport untuk fungsionalitas Composable. `androidx.compose.ui` diimport untuk modifier UI seperti Modifier, Alignment, draw.clip, layout.ContentScale, platform.LocalContext, dan `text.font.FontWeight`, `text.style.TextOverflow`. `androidx.compose.ui.unit.dp` diimport untuk unit densitas piksel. `AsyncImage` dan `ImageRequest` dari pustaka Coil diimport untuk memuat gambar secara asinkron. `Movie` diimport sebagai model domain film.

Pada baris [21], anotasi `@OptIn(ExperimentalMaterial3Api::class)` digunakan untuk mengizinkan penggunaan API eksperimental dari Material3, seperti `CardDefaults.cardElevation`.

Pada baris [22], fungsi MovieCard dideklarasikan sebagai Composable. Fungsi ini menerima objek Movie, sebuah modifier opsional, sebuah lambda onMovieClick yang akan dipanggil ketika kartu film diklik, dan sebuah lambda onTmdbClick yang akan dipanggil ketika tombol TMDB diklik.

Pada baris [27], variabel context dideklarasikan dan diinisialisasi dengan LocalContext.current, yang menyediakan akses ke konteks Android saat ini, berguna untuk membangun permintaan gambar Coil.

Pada baris [29–35], komponen Card dideklarasikan sebagai elemen dasar untuk menampilkan informasi film. modifier diterapkan untuk mengatur lebar kartu, membuatnya dapat diklik (clickable { onMovieClick(movie) }), dan memberikan padding. shape = RoundedCornerShape(12.dp) memberikan sudut membulat pada kartu, dan elevation = CardDefaults.cardElevation(defaultElevation = 4.dp) memberikan efek bayangan yang mengangkat kartu dari latar belakang.

Pada baris [36–69], Row dideklarasikan di dalam Card untuk mengatur tata letak horizontal konten film. modifier disetel untuk mengisi lebar penuh dan memberikan padding.

Pada baris [40–47], AsyncImage digunakan untuk memuat dan menampilkan gambar poster film secara asinkron. model dibangun menggunakan ImageRequest.Builder dengan konteks lokal dan URL poster film yang didapatkan dari movie.getPosterUrl(). crossfade(true) menambahkan efek fade-in saat gambar dimuat. contentDescription diatur ke judul film untuk aksesibilitas. modifier mengatur ukuran gambar, memberinya sudut membulat, dan contentScale = ContentScale.Crop memastikan gambar dipangkas agar sesuai dengan batasnya.

Pada baris [49], Spacer dideklarasikan untuk memberikan ruang horizontal sebesar 12.dp antara gambar poster dan detail teks.

Pada baris [51–68], Column dideklarasikan untuk mengatur tata letak vertikal detail teks film. modifier memberikan bobot 1f agar kolom ini mengambil sisa ruang horizontal dan align(Alignment.CenterVertically) menyejajarkan konten kolom secara vertikal di tengah.

Pada baris [53–57], Text digunakan untuk menampilkan judul film. text disetel ke movie.title, style menggunakan MaterialTheme.typography.titleMedium, fontWeight disetel ke Bold, maxLines = 2 membatasi judul hingga dua baris, dan overflow = TextOverflow.Ellipsis menambahkan elipsis jika judul terlalu panjang.

Pada baris [58], Spacer dideklarasikan untuk memberikan ruang vertikal sebesar 4.dp.

Pada baris [60–62], Text digunakan untuk menampilkan rating film, diformat menjadi satu desimal. style menggunakan `MaterialTheme.typography.bodySmall`.

Pada baris [63], Spacer dideklarasikan untuk memberikan ruang vertikal sebesar 8.dp.

Pada baris [65–68], Row dideklarasikan untuk menampung tombol-tombol aksi, dengan `horizontalArrangement = Arrangement.End` untuk menempatkan tombol di sisi kanan.

Pada baris [69–72], Button pertama dideklarasikan. `onClick` diatur untuk memanggil `onTmdbClick` dengan URL TMDb film yang didapatkan dari `movie.getTmdbUrl()`. modifier memberikan padding di sisi kanan. `Text("TMDb")` ditampilkan sebagai label tombol.

Pada baris [74–76], Button kedua dideklarasikan. `onClick` diatur untuk memanggil `onMovieClick` dengan objek `movie`, memungkinkan navigasi ke detail film. `Text("Detail")` ditampilkan sebagai label tombol.

presentation/ui/navigation/Screen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], dideklarasikan sealed class `Screen` dengan sebuah properti `route` bertipe `String`. Sealed class ini digunakan untuk merepresentasikan semua kemungkinan tujuan (destinasi) dalam navigasi aplikasi secara aman dan type-safe. Properti `route` akan digunakan oleh Jetpack Compose Navigation untuk mengidentifikasi layar yang berbeda.

Pada baris [4], `MovieList` dideklarasikan sebagai data object yang merupakan turunan dari `Screen`. Ini merepresentasikan layar daftar film dan memiliki route `"movie_list"`.

Pada baris [5], `MovieDetail` dideklarasikan sebagai data object yang juga merupakan turunan dari `Screen`. Ini merepresentasikan layar detail film dan memiliki route `"movie_detail"`. Penggunaan data object adalah cara yang efisien untuk mendefinisikan objek tunggal (singleton) dalam Kotlin, yang sangat cocok untuk merepresentasikan rute navigasi yang tidak memerlukan parameter tambahan dalam definisi rutenya.

presentation/ui/screens/detail/MovieDetailScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–19], `import` berbagai komponen yang diperlukan dari Jetpack Compose dan pustaka lainnya. `androidx.compose.foundation` `import` untuk elemen UI dasar seperti tata

letak, scrolling, dan state. `androidx.compose.material.icons.Icons` dan `androidx.compose.material.icons.filled.Star` diimport untuk ikon bintang. `androidx.compose.material3` diimport untuk komponen Material Design 3 seperti Text, Icon, MaterialTheme, dan Scaffold (meskipun Scaffold tidak digunakan secara langsung di Composable ini, komponen Material3 lainnya digunakan). `androidx.compose.runtime` diimport untuk fungsionalitas Composable. `androidx.compose.ui` diimport untuk modifier UI seperti Modifier, Alignment, draw.clip, graphics.Color, layout.ContentScale, platform.LocalContext, text.font.FontWeight, dan unit.dp untuk unit densitas piksel. AsyncImage dan ImageRequest dari pustaka Coil diimport untuk memuat gambar secara asinkron. Movie diimport sebagai model domain film.

Pada baris [21], fungsi MovieDetailScreen dideklarasikan sebagai Composable. Fungsi ini menerima objek movie yang berisi detail film yang akan ditampilkan.

Pada baris [22–27], Column dideklarasikan sebagai tata letak vertikal utama untuk seluruh konten layar detail film. modifier diterapkan untuk mengisi seluruh ukuran layar (fillMaxSize()), menambahkan padding 16.dp di sekelilingnya, dan membuatnya dapat di-scroll secara vertikal (verticalScroll(rememberScrollState())) jika konten melebihi tinggi layar.

Pada baris [28–36], AsyncImage digunakan untuk memuat dan menampilkan gambar backdrop film secara asinkron. model dibangun menggunakan ImageRequest.Builder dengan konteks lokal dan URL backdrop film yang didapatkan dari movie.getBackdropUrl(). crossfade(true) menambahkan efek fade-in saat gambar dimuat. contentDescription diatur ke judul film untuk aksesibilitas. modifier mengatur gambar untuk mengisi lebar penuh, memiliki tinggi 220.dp, dan memiliki sudut membulat sesuai bentuk tema Material (clip(MaterialTheme.shapes.medium)), sementara contentScale = ContentScale.Crop memastikan gambar dipangkas agar sesuai dengan batasnya.

Pada baris [37], Spacer dideklarasikan untuk memberikan ruang vertikal sebesar 16.dp.

Pada baris [39–42], Text digunakan untuk menampilkan judul film. text disetel ke movie.title, style menggunakan MaterialTheme.typography.headlineMedium, dan fontWeight disetel ke FontWeight.Bold untuk menonjolkan judul.

Pada baris [43], Spacer dideklarasikan untuk memberikan ruang vertikal sebesar 8.dp.

Pada baris [45–56], Row dideklarasikan untuk mengatur tata letak horizontal ikon bintang dan rating. `verticalAlignment = Alignment.CenterVertically` memastikan elemen di tengah secara vertikal.

Pada baris [46–51], Icon digunakan untuk menampilkan ikon bintang. `imageVector` disetel ke `Icons.Filled.Star`, `contentDescription` diatur sebagai "Rating" untuk aksesibilitas, tint disetel ke warna kuning (`Color(0xFFFFDD835)`), dan modifier mengatur ukuran ikon menjadi 24.dp.

Pada baris [52], Spacer dideklarasikan untuk memberikan ruang horizontal sebesar 4.dp.

Pada baris [53–56], Text digunakan untuk menampilkan rating film yang diformat menjadi satu desimal dari `movie.voteAverage`. style menggunakan `MaterialTheme.typography.titleMedium`.

Pada baris [57], Spacer dideklarasikan untuk memberikan ruang vertikal sebesar 8.dp.

Pada baris [59–62], blok `movie.releaseDate?.let { ... }` digunakan untuk menampilkan tanggal rilis film hanya jika `releaseDate` tidak null. Di dalamnya, Text menampilkan tanggal rilis, dan Spacer memberikan ruang vertikal sebesar 8.dp setelahnya.

Pada baris [64–67], Text digunakan untuk menampilkan deskripsi atau overview film. text disetel ke `movie.overview`, dan style menggunakan `MaterialTheme.typography.bodyLarge`.

presentation/ui/screens/list/MovieListScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–24], diimport berbagai komponen yang diperlukan dari Android, Jetpack Compose, dan pustaka lainnya. Intent dan Uri diimport untuk tujuan deep linking atau membuka URL eksternal. `androidx.compose.foundation.layout` diimport untuk komponen tata letak. `androidx.compose.foundation.lazy.LazyColumn`, `LazyVerticalGrid`, `GridCells`, dan `items` diimport untuk menampilkan daftar item yang efisien. Ikon-ikon `Icons.Filled.CheckCircle`, `Icons.Filled.Menu`, dan `Icons.AutoMirrored.Filled.List` diimport untuk tombol action di top bar. `androidx.compose.material3` diimport untuk komponen Material Design 3 seperti Scaffold, TopAppBar, Text, IconButton, Icon, CircularProgressIndicator, dan ExperimentalMaterial3Api. `androidx.compose.runtime` diimport untuk fungsionalitas Composable dan state management seperti `rememberCoroutineScope`, `collectAsState`, dan `getValue`. `androidx.compose.ui` diimport

untuk Modifier dan Alignment. `androidx.compose.ui.platform.LocalContext` diimport untuk mendapatkan konteks Android. `androidx.compose.ui.unit.dp` diimport untuk unit densitas piksel. `NavController` dari `androidx.navigation.compose` diimport untuk navigasi. `MovieCard` diimport sebagai komponen UI kustom. `LayoutType` dan `SettingsManager` diimport untuk pengelolaan pengaturan aplikasi. `Gson` diimport untuk serialisasi objek. `launch` dari `kotlinx.coroutines` diimport untuk meluncurkan coroutine.

Pada baris [26], anotasi `@OptIn(ExperimentalMaterial3Api::class)` digunakan untuk mengizinkan penggunaan API eksperimental dari Material3, seperti `TopAppBar`.

Pada baris [27], fungsi `MovieListScreen` dideklarasikan sebagai `Composable`. Fungsi ini menerima `viewModel` (instance `MovieListViewModel`), `settingsManager` (instance `SettingsManager`), dan `navController` (instance `NavController`) sebagai parameter.

Pada baris [33], variabel `uiState` diobservasi dari `viewModel.uiState.collectAsState()`, yang akan memicu rekomposisi UI setiap kali status UI berubah.

Pada baris [34], variabel `layoutType` diobservasi dari `settingsManager.layoutType.collectAsState()`, dengan nilai awal `LayoutType.LIST`. Ini akan mengontrol apakah film ditampilkan dalam tata letak daftar atau grid.

Pada baris [35], variabel `isDarkMode` diobservasi dari `settingsManager.isDarkMode.collectAsState()`, dengan nilai awal `false`. Ini akan mengontrol tema aplikasi.

Pada baris [36], variabel `context` diinisialisasi dengan `LocalContext.current` untuk mendapatkan konteks Android saat ini, yang diperlukan untuk memulai Intent.

Pada baris [37], variabel `scope` diinisialisasi dengan `rememberCoroutineScope()`, yang menyediakan `CoroutineScope` yang terkait dengan siklus hidup `Composable`, digunakan untuk meluncurkan coroutine.

Pada baris [39–98], `Scaffold` dideklarasikan sebagai komponen Material Design 3 yang menyediakan slot untuk top bar, bottom bar, snackbar, dan konten utama.

Pada baris [40–64], blok `topBar` dari `Scaffold` didefinisikan.

Pada baris [41–63], `TopAppBar` dideklarasikan sebagai bilah atas aplikasi. `title` disetel ke `Text("Popular Movies")`.

Pada baris [42–62], blok `actions` dari `TopAppBar` didefinisikan untuk menampung tombol aksi di sisi kanan.

Pada baris [43–50], IconButton pertama dideklarasikan. Ketika diklik, ia meluncurkan coroutine menggunakan scope.launch yang kemudian memanggil settingsManager.toggleDarkMode() untuk mengubah tema aplikasi antara terang dan gelap. Icon ditampilkan dengan ikon CheckCircle yang sama untuk kedua mode (meskipun secara umum ikon akan berubah untuk menunjukkan status).

Pada baris [52–61], IconButton kedua dideklarasikan. Ketika diklik, ia meluncurkan coroutine menggunakan scope.launch yang kemudian menghitung newLayout (mengubah dari LIST ke GRID atau sebaliknya) dan memanggil settingsManager.setLayoutType(newLayout) untuk mengubah tata letak film. Icon ditampilkan dengan ikon Menu jika tata letak saat ini LIST dan List (dari AutoMirrored.Filled) jika tata letak saat ini GRID.

Pada baris [64], paddingValues adalah parameter yang disediakan oleh Scaffold untuk memastikan konten tidak tumpang tindih dengan top bar.

Pada baris [65–97], Box dideklarasikan sebagai wadah untuk konten utama layar, dengan modifier mengisi seluruh ruang dan menerapkan paddingValues dari Scaffold.

Pada baris [67–68], kondisi if (uiState.isLoading && uiState.movies.isEmpty()) memeriksa apakah aplikasi sedang memuat data dan daftar film masih kosong. Jika ya, CircularProgressIndicator ditampilkan di tengah layar.

Pada baris [69–75], kondisi else if (uiState.errorMessage != null && uiState.movies.isEmpty()) memeriksa apakah ada pesan error dan daftar film masih kosong. Jika ya, Text ditampilkan dengan pesan error (atau "Unknown Error" jika pesan null), dengan warna error, di tengah layar.

Pada baris [76–96], blok else dieksekusi jika aplikasi tidak dalam kondisi loading atau error yang membuat daftar kosong, yang berarti data film siap ditampilkan.

Pada baris [77–95], kondisi if (layoutType == LayoutType.LIST) memeriksa jenis tata letak yang dipilih.

Pada baris [78–90], jika layoutType adalah LIST, LazyColumn digunakan untuk menampilkan daftar film secara vertikal. modifier mengisi seluruh ukuran, contentPadding memberikan padding di sekeliling daftar, dan verticalArrangement memberikan jarak antar item.

Pada baris [81–89], `items(uiState.movies)` mengulang melalui daftar film dan untuk setiap film, `MovieCard` ditampilkan. `modifier` diatur untuk mengisi lebar penuh. `onMovieClick` lambda mendefinisikan navigasi ke layar detail film: objek `selectedMovie` di-serialisasi ke JSON, di-encode URL, lalu `navController.navigate` dipanggil dengan rute `"movie_detail/"` diikuti oleh JSON film. `onTmdbClick` lambda mendefinisikan aksi untuk membuka URL TMDB di browser eksternal: sebuah `Intent` dibuat dengan `ACTION_VIEW` dan `Uri` yang diparse dari `tmdbUrl`, lalu `context.startActivity` dipanggil.

Pada baris [91–95], jika `layoutType` adalah `GRID`, `LazyVerticalGrid` digunakan untuk menampilkan daftar film dalam tata letak grid dengan 2 kolom tetap. `modifier` mengisi seluruh ukuran, `contentPadding` memberikan padding, dan `verticalArrangement` serta `horizontalArrangement` memberikan jarak antar item.

Pada baris [96–104], `items(uiState.movies)` mengulang melalui daftar film dalam grid, dan untuk setiap film, `MovieCard` ditampilkan dengan logika `onMovieClick` dan `onTmdbClick` yang sama seperti pada `LazyColumn`.

presentation/ui/screens/list/MovieListViewModel.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–10], diimport berbagai komponen yang diperlukan dari pustaka Jetpack AndroidX, Kotlin Coroutines, dan model data aplikasi. `ViewModel` dan `viewModelScope` diimport untuk fungsionalitas `ViewModel` dan manajemen coroutine. `NetworkResult` diimport untuk merepresentasikan status hasil jaringan. `Movie` diimport sebagai model domain film. `GetMoviesUseCase` diimport sebagai use case untuk mendapatkan data film. `MutableStateFlow`, `StateFlow`, `asStateFlow`, `catch`, `onStart`, dan `launch` diimport dari `kotlinx.coroutines.flow` dan `kotlinx.coroutines` untuk manajemen aliran data reaktif dan konkurensi.

Pada baris [12–16], kelas data `MovieListUiState` dideklarasikan. Kelas ini merepresentasikan status UI dari layar daftar film. Ini mencakup `movies` (daftar objek `Movie` yang ditampilkan, default kosong), `isLoading` (boolean yang menunjukkan apakah data sedang dimuat, default false), dan `errorMessage` (string opsional untuk pesan error, default null).

Pada baris [18], kelas `MovieListViewModel` dideklarasikan sebagai turunan dari `ViewModel`. `ViewModel` ini bertanggung jawab untuk menyiapkan data untuk UI,

menangani logika bisnis, dan berinteraksi dengan use case dan repositori. Konstruktornya menerima `getMoviesUseCase` sebagai dependensinya.

Pada baris [22], variabel `_uiState` dideklarasikan sebagai `MutableStateFlow` yang menyimpan instance `MovieListUiState` dengan nilai awal default. `MutableStateFlow` adalah aliran data yang dapat diubah dan memegang nilai terakhir yang dipancarkan.

Pada baris [23], variabel `uiState` dideklarasikan sebagai `StateFlow` yang merupakan versi read-only dari `_uiState` (`_uiState.asStateFlow()`). Ini adalah aliran yang akan diobservasi oleh UI Composable.

Pada baris [25–28], blok `init` dideklarasikan. Blok ini akan dieksekusi segera setelah instance `MovieListViewModel` dibuat.

Pada baris [26], `observeMovies()` dipanggil untuk memulai pengumpulan data film dari database lokal.

Pada baris [27], `refreshData()` dipanggil untuk memicu pengambilan data film terbaru dari API.

Pada baris [30–44], fungsi privat `observeMovies()` dideklarasikan. Fungsi ini bertanggung jawab untuk mengamati aliran data film dari database lokal.

Pada baris [31], `viewModelScope.launch` digunakan untuk meluncurkan coroutine dalam cakupan `ViewModel`, memastikan coroutine dibatalkan ketika `ViewModel` dihapus.

Pada baris [32], `getMoviesUseCase.getMoviesFromDb()` dipanggil untuk mendapatkan `Flow` dari daftar film dari database.

Pada baris [33], `.onStart { _uiState.value = _uiState.value.copy(isLoading = true) }` adalah operator aliran yang akan dieksekusi sebelum aliran mulai mengumpulkan item. Ini mengatur `isLoading` di `_uiState` menjadi `true` untuk menunjukkan bahwa proses pemuatan dimulai.

Pada baris [34–38], `.catch { e -> ... }` adalah operator aliran yang akan menangkap pengecualian apa pun yang terjadi di dalam aliran. Jika error terjadi saat memuat dari database, `isLoading` diatur menjadi `false` dan `errorMessage` diatur dengan pesan error yang sesuai.

Pada baris [39–43], `.collect { movies -> ... }` adalah operator yang mengumpulkan item yang dipancarkan oleh aliran. Setiap kali daftar film dari database berubah, `isLoading` diatur menjadi `false` dan `movies` di `_uiState` diperbarui dengan daftar film yang baru.

Pada baris [46–63], fungsi `refreshData()` dideklarasikan. Fungsi ini bertanggung jawab untuk mengambil data film terbaru dari API.

Pada baris [47], `viewModelScope.launch` digunakan untuk meluncurkan coroutine.

Pada baris [48], `_uiState.value = _uiState.value.copy(isLoading = true, errorMessage = null)` mengatur `isLoading` menjadi `true` dan menghapus `errorMessage` sebelumnya untuk menunjukkan bahwa proses pembaruan data dimulai.

Pada baris [49], `when (val result = getMoviesUseCase.refreshMoviesFromApi())` memanggil use case untuk menyegarkan data dari API dan kemudian memproses hasilnya berdasarkan jenis `NetworkResult`.

Pada baris [50–52], jika `result` adalah `NetworkResult.Success`, `isLoading` diatur kembali ke `false`.

Pada baris [53–57], jika `result` adalah `NetworkResult.Error`, `isLoading` diatur ke `false` dan `errorMessage` diatur ke pesan yang diterima dari `result`.

Pada baris [58], `else -> {}` menangani kasus lain dari `NetworkResult` (misalnya, `Loading`, meskipun biasanya `Loading` tidak akan diproses di sini karena `refreshMoviesFromApi()` segera mengembalikan `Success` atau `Error`).

presentation/ui/screens/list/MovieListViewModelFactory.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–5], diimport komponen yang diperlukan dari pustaka Jetpack AndroidX. `ViewModel` dan `ViewModelProvider` diimport untuk fungsionalitas `ViewModel` dan kemampuan untuk membuat instance `ViewModel`. `GetMoviesUseCase` diimport sebagai use case yang akan disuntikkan ke dalam `ViewModel`.

Pada baris [7], kelas `MovieListViewModelFactory` dideklarasikan. Kelas ini mengimplementasikan antarmuka `ViewModelProvider.Factory`. Pola Factory ini diperlukan ketika `ViewModel` memiliki dependensi (dalam hal ini, `GetMoviesUseCase`) yang perlu disediakan pada saat pembuatan.

Konstruktor kelas ini menerima sebuah instance `getMoviesUseCase` sebagai parameter privat. Ini adalah dependensi yang akan diberikan kepada `MovieListViewModel` ketika dibuat.

Pada baris [11–17], fungsi `create()` di-override dari antarmuka `ViewModelProvider.Factory`. Fungsi ini bertanggung jawab untuk membuat dan mengembalikan instance `ViewModel` yang sesuai.

Pada baris [12], `if (modelClass.isAssignableFrom(MovieListViewModel::class.java))` memeriksa apakah `modelClass` yang diminta (yaitu, kelas `ViewModel` yang ingin dibuat) adalah `MovieListViewModel` atau salah satu subclass-nya.

Pada baris [14], jika kelas yang diminta adalah `MovieListViewModel`, `MovieListViewModel(getMoviesUseCase)` dipanggil untuk membuat instance baru dari `MovieListViewModel`, dengan menyuntikkan `getMoviesUseCase` yang telah diterima oleh factory ini.

Pada baris [15], `@SuppressWarnings("UNCHECKED_CAST")` digunakan untuk menekan peringatan kompilasi karena casting dari `MovieListViewModel` ke `T` tidak dapat diperiksa secara ketat pada waktu kompilasi. Ini aman karena pemeriksaan `isAssignableFrom` sudah memastikan jenis yang benar.

Pada baris [17], jika `modelClass` yang diminta bukan `MovieListViewModel`, `throw IllegalArgumentException("Unknown ViewModel class")` akan dilemparkan, mengindikasikan bahwa factory ini tidak tahu cara membuat `ViewModel` untuk kelas yang diminta.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

[Praktikum_Mobile/Modul 5 at main · Bukhrii/Praktikum_Mobile](#)